Paper Review • arXiv 2610.35850

# OSWAM: “Imagination Tokens” Make World Action Models Act in One Step

OSWAM: An Effective World Action Model with One-Step Action Generation

Guowen Zhang*, Liyi Chen*, Xu Hu, Pengfei Wang, Jingfan Chen, Chenhang He, Lei Zhang †
PolyU Visual Computing Lab • First submitted 2026-10-26

Driving • NAVSIM / nuScenes

Bimanual • RoboTwin 2.0

Real robot • PiPER arm

TL;DR

# Structured prior replaces pure noise: 10 → 1 denoising step, accuracy kept

**OSWAM** adds a few "imagination tokens" (AIT / SIT) to the Video DiT, yielding a structured action prior in one forward pass that replaces Gaussian noise as the Action DiT starting point; feature distillation from a teacher that sees future frames lets actions need just **one denoising step**.

**10 → 1**

**Action denoising steps**

Direct-policy WAMs (Fast-WAM / SimWAM) need ~10 steps

**≈3×**

**End-to-end speedup**

NAVSIM 136 ms vs SimWAM 410 ms (H20)

**90.3 / 90.0**

**PDMS / RoboTwin success**

SimWAM 90.3; Fast-WAM 91.83 (− 1.83)

**40.8 ms**

**Compiled latency**

RoboTwin, RTX 5090, torch.compile + CUDA Graph

Positioning: keeps Fast-WAM's direct-policy framework and tackles only the remaining multi-step action denoising bottleneck.

# 3 WAM types: direct policies skip video, yet need ~10 action steps

## Imagine, then act

Inverse-dynamics WAM

$p(a|o,l) = \int p(v|o,l) \cdot p(a|o,l,v)\, dv$

Generates future video at inference, then infers actions

| | |
|---|---|
| LingBot-VA | **7659 ms** |
| DriveWAM | **1032 ms** |

## Joint-WAM

Joint video-action generation

$p(a, v \mid o, l)$

Video and actions denoised jointly in one generative process

| | |
|---|---|
| Motus | **2367 ms** |

## Direct-policy WAM

Fast-WAM / SimWAM

$p(a|o,l) = p(a \mid z(o))$

Video prediction only as training signal; inference encodes the first frame only

| | |
|---|---|
| Fast-WAM | **397 ms** |
| SimWAM | **410 ms** |

**Remaining bottleneck:** direct policies still need ~10 flow-matching action denoising steps ($K \approx 10$) → OSWAM's goal: **$K = 1$**



Source: Sec. 3.1, Tables 1/2 (latency measured on H20)

Problem

# Cutting straight to one step fails: random noise lacks action structure

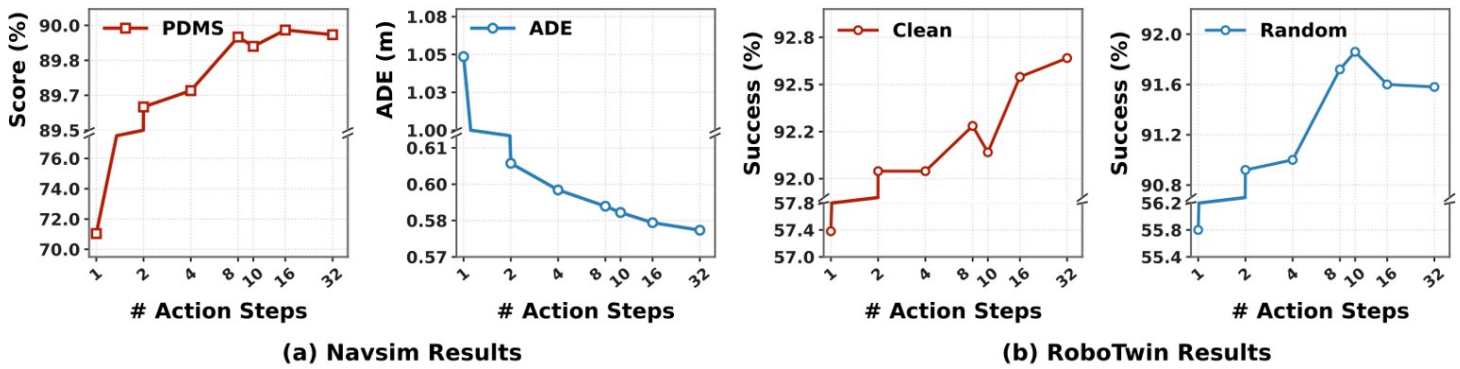



Fig. 2: Fast-WAM vs. number of action denoising steps (left: NAVSIM PDMS / ADE; right: RoboTwin Clean / Random)

**↓ ≈20%**

**NAVSIM: PDMS drops at 1 step**

**↓ ≈40%**

**RoboTwin: success drops at 1 step**

**Appendix Table A.4: baselines at 1 step**

| Method (1 step) | Metric | Result |
|---|---|---|
| SimWAM | NAVSIM PDMS | 68.9 |
| DriveVA | NAVSIM PDMS | 13.2 |
| Fast-WAM | RoboTwin avg. | 56.59% |

**Hypothesis:** one step must recover a coherent trajectory from pure noise; step distillation / rectified flow still start from Gaussian noise → **a structured initialization is needed**

Key Idea

# New start: from “pure noise + 10 steps” to “structured prior + 1-step fix”

**Direct-policy WAM**

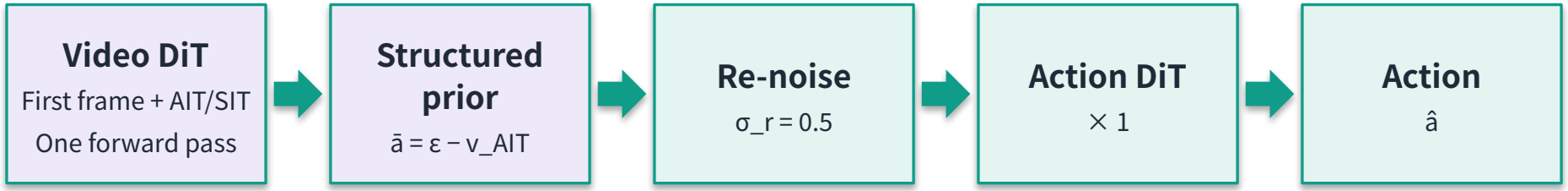
Fast-WAM / SimWAM




Action GPU time 67.61 ms (Fast-WAM, RoboTwin, RTX 5090)

**OSWAM**

Ours

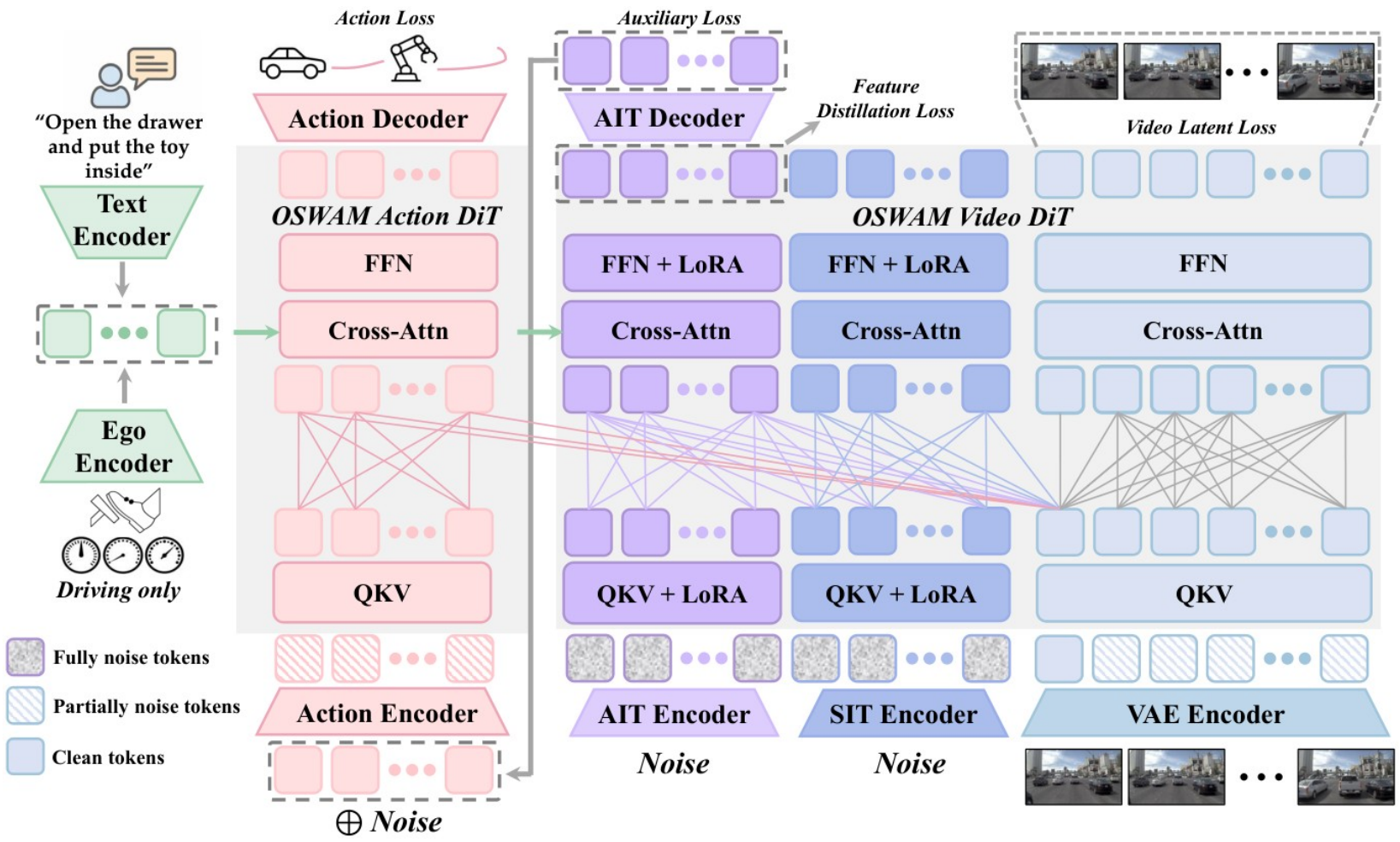



**Action GPU time 6.78 ms; Video DiT adds only 5.48 ms**

**One-step update (Eq. 4): $\hat{a} = a_t - t \cdot v_\theta(a_t, t)$**

From pure noise (t = 1), one update must span the whole denoising path, often giving coarse actions; OSWAM starts from a structured intermediate state with t < 1, so the Action DiT only corrects the residual.

# One Video DiT pass yields first-frame features and an action prior




### Action DiT (pink)

Starts from the AIT prior, one denoising step; supervised by Action Loss

### AIT / SIT (purple / blue)

Noise-initialized imagination tokens, LoRA-adapted; AIT has auxiliary and distillation losses, SIT none

### Video tokens (light blue)

Clean first frame, noised future frames; future used only for training Video Latent Loss

### Conditioning

T5 text + ego encoder (driving only) via cross-attention; Video and Action share MoT attention

# AIT regresses a coarse action, re-noised for a one-step Action DiT fix

1

## AIT: Action Imagination Tokens

Encode action noise ε, t_max = 1000; attend to first frame + SIT + AIT → auxiliary decoder gives prior ā (L_AIT)

2

## SIT: Scene Imagination Tokens

Separately noise-initialized; attend to first frame + SIT only; scene-context slots with no direct loss, learned via AIT attention

3

## Re-noise

Stop-gradient sg(ā), mixed with the same ε to σ_r = 0.5: keeps prior structure, leaves room to refine

4

## Action DiT one-step refinement

Predicts v_act conditioned on first-frame features and task; target scaled by 1/σ_r so one Euler step reaches ground truth

## Key Equations

AIT prior

$$\bar{a} = \varepsilon - v_{AIT}$$

Re-noising (Eq. 5)

$$a_{init} = (1-\sigma_r) \cdot sg(\bar{a}) + \sigma_r \cdot \varepsilon$$

One-step denoising (Eq. 6)

$$\hat{a} = a_{init} - \sigma_r \cdot v_{act}$$

Training target (Eq. 11)

$$v_{act} \rightarrow (a_{init} - a_{gt}) / \sigma_r$$

Total loss (Eq. 12)

$$L = L_{vid} + L_{AIT} + \lambda \cdot L_{dist} + L_{act}$$

t_max = 1000 , τ_r = 500 （σ_r = 0.5） , λ_dist = 10

# Masks enable first-frame inference; role-aware RoPE splits 3 token types

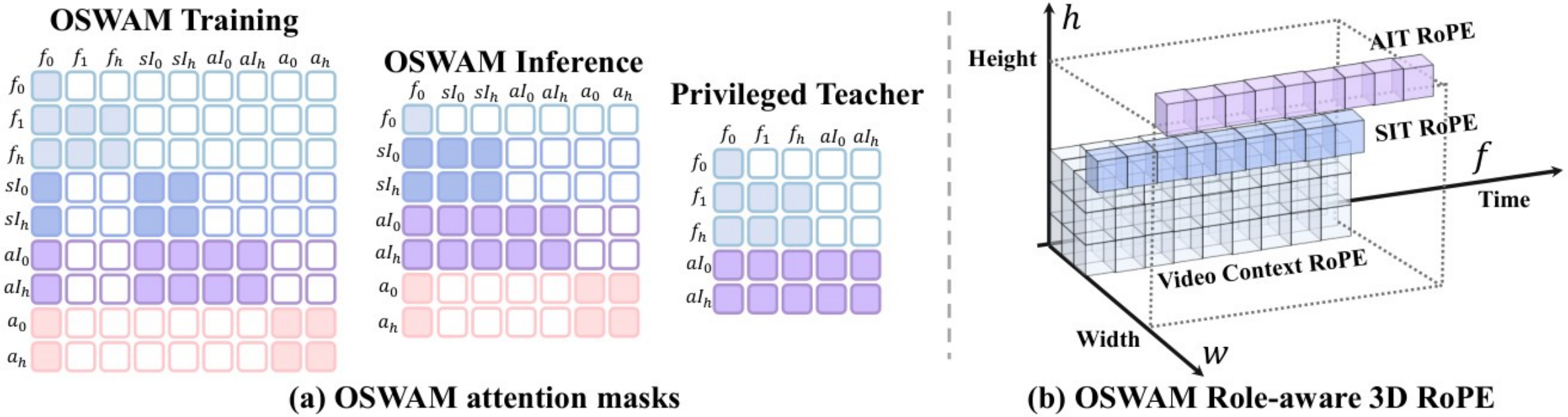



Fig. 4: (a) Attention masks for OSWAM training / inference and privileged teacher; (b) Role-aware 3D RoPE (sI = SIT, aI = AIT)

## Attention masks

SIT → first frame + SIT; AIT → first frame + SIT + AIT; action → first frame + action

SIT / AIT / action tokens never use future frames, so these are dropped at inference; teacher AIT sees the full clip

## Role-aware 3D RoPE

Video: (t, h, w); AIT: (t_n, H_v, W_v), off-grid; SIT: (t_n, grid center)

t_n = n・s_a / (s_v・d_t): maps action positions to video latent time

# Teacher sees future clip, student only first frame: zero inference cost

## Privileged teacher (frozen)

Video DiT + AIT; AIT attends to the full future clip $f_0 \ldots f_h$

GT future latents noised at $\tau_v = 200$; first frame kept clean

**Layer-wise AIT feature alignment (L2)**

Video DiT blocks 8 / 16 / 24 / 30

## OSWAM student

AIT sees only first frame + SIT + AIT; SIT supervised indirectly via attention

Teacher dropped at inference → same first-frame interface, no extra compute

## Why distill?

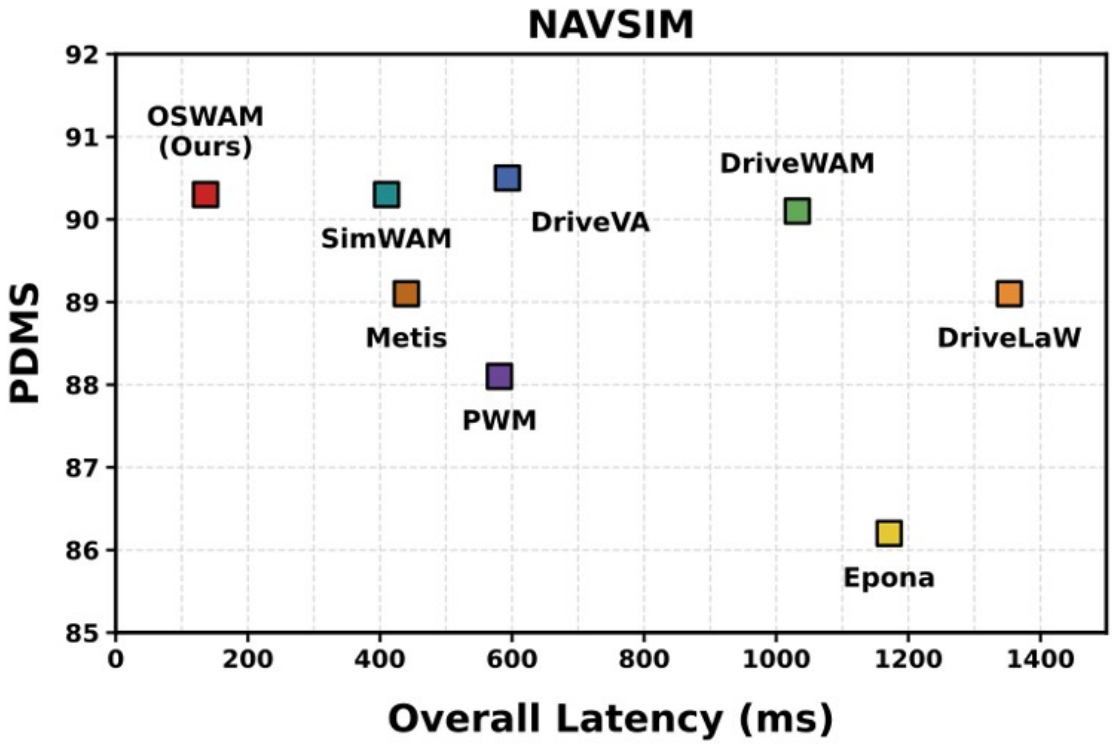
With only the first frame, long-horizon information must be implicitly compressed into its representation; transferring a future-aware teacher's features to AIT strengthens future-oriented action generation.

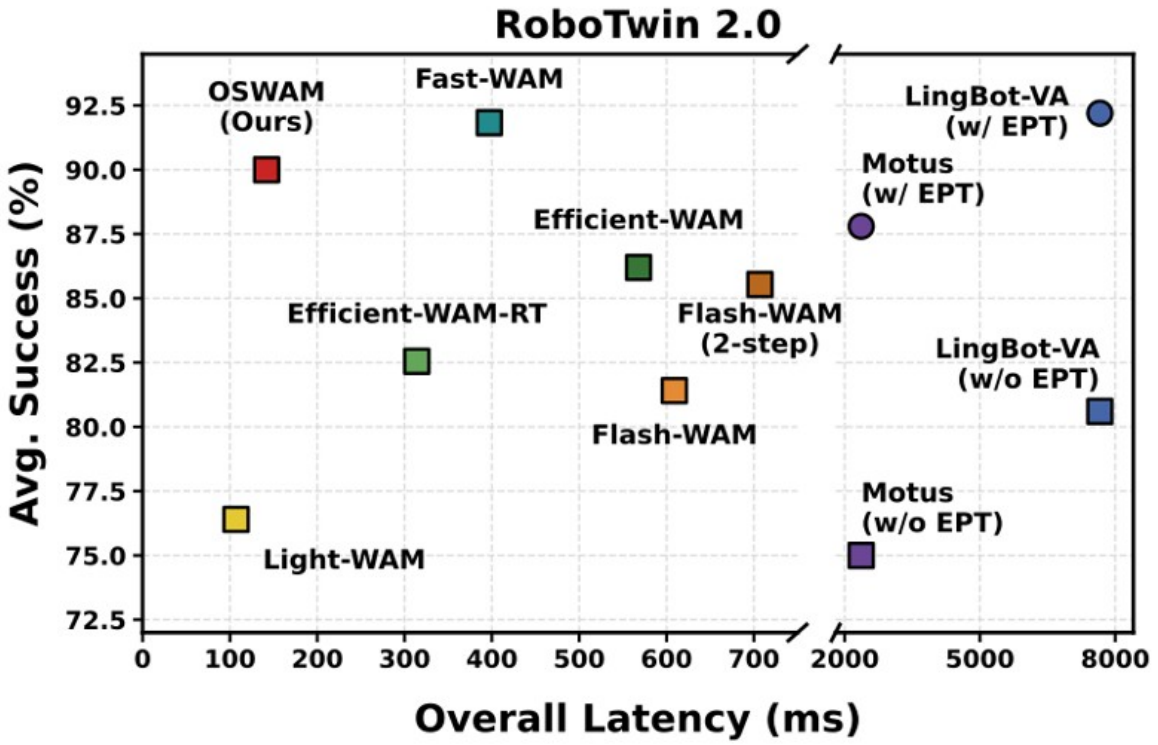
$$L_{dist} = |J|^{-1} \cdot \Sigma_j \| h_j^{stu} - sg(h_j^{tea}) \|^2$$

- Teacher and student share video and action noise samples
- $\lambda_{dist} = 10$; distillation starts at step 500, linear warm-up over 500 steps
- Teacher trained like OSWAM; only token groups and masks differ

**Ablation gains (Table 4):** NAVSIM +0.15 PDMS; RoboTwin Clean +2.0 / Rand. +1.5

# OSWAM is top-left on speed vs. performance in both benchmarks




**NAVSIM：** 136 ms / 90.3 （SimWAM 410 ms / 90.3）

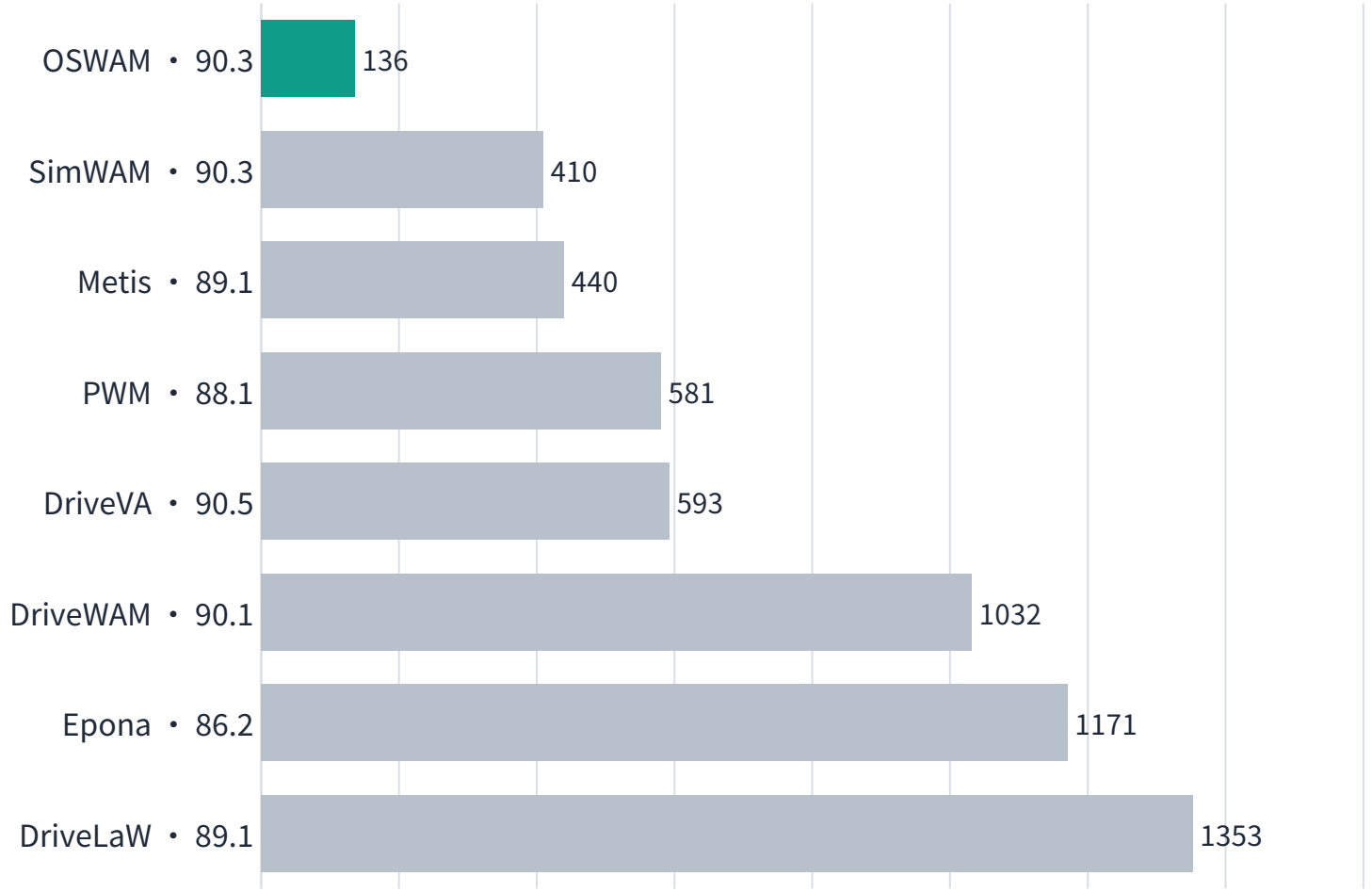
**RoboTwin：** 142 ms / 90.0% （Fast-WAM 397 ms / 91.83%）



Source: Fig. 1 (latency measured on NVIDIA H20)

# NAVSIM: PDMS 90.3 matches SimWAM, latency 410 → 136 ms

Overall latency (ms, lower is better); labels: method • PDMS




**3.01×**

**vs SimWAM (10 steps)**

Same PDMS of 90.3

**3.24× +1.2**

**vs Metis (10 steps)**

PDMS 90.3 vs 89.1

**4.36× −0.2**

**vs DriveVA (2 video + 2 action steps)**

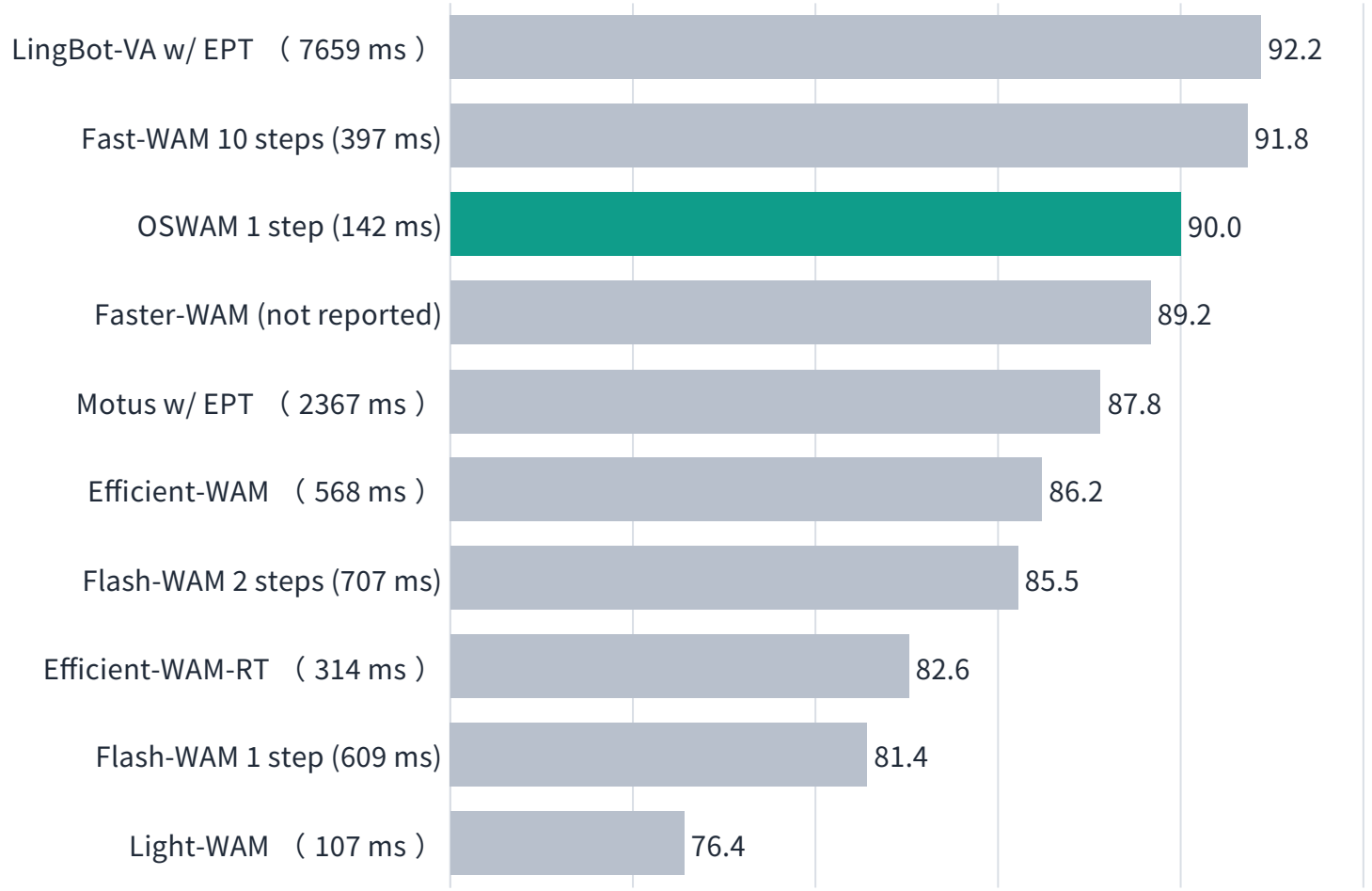
PDMS 90.3 vs 90.5



Source: Table 1 (NAVSIM navtest, H20; Metis, DriveVA latency reproduced by authors)

# RoboTwin 2.0: only 1.83 pts below 10-step Fast-WAM, 2.80× faster

Avg. success rate (%, higher is better); latency in parentheses; EPT = embodied pretraining




**−1.83 / 2.80×**

**vs Fast-WAM (10 steps)**

90.00% vs 91.83%

**+8.59 / 4.29×**

**vs Flash-WAM (1 step)**

90.00% vs 81.41%

**+13.6 / +35 ms**

**vs Light-WAM (single pass)**

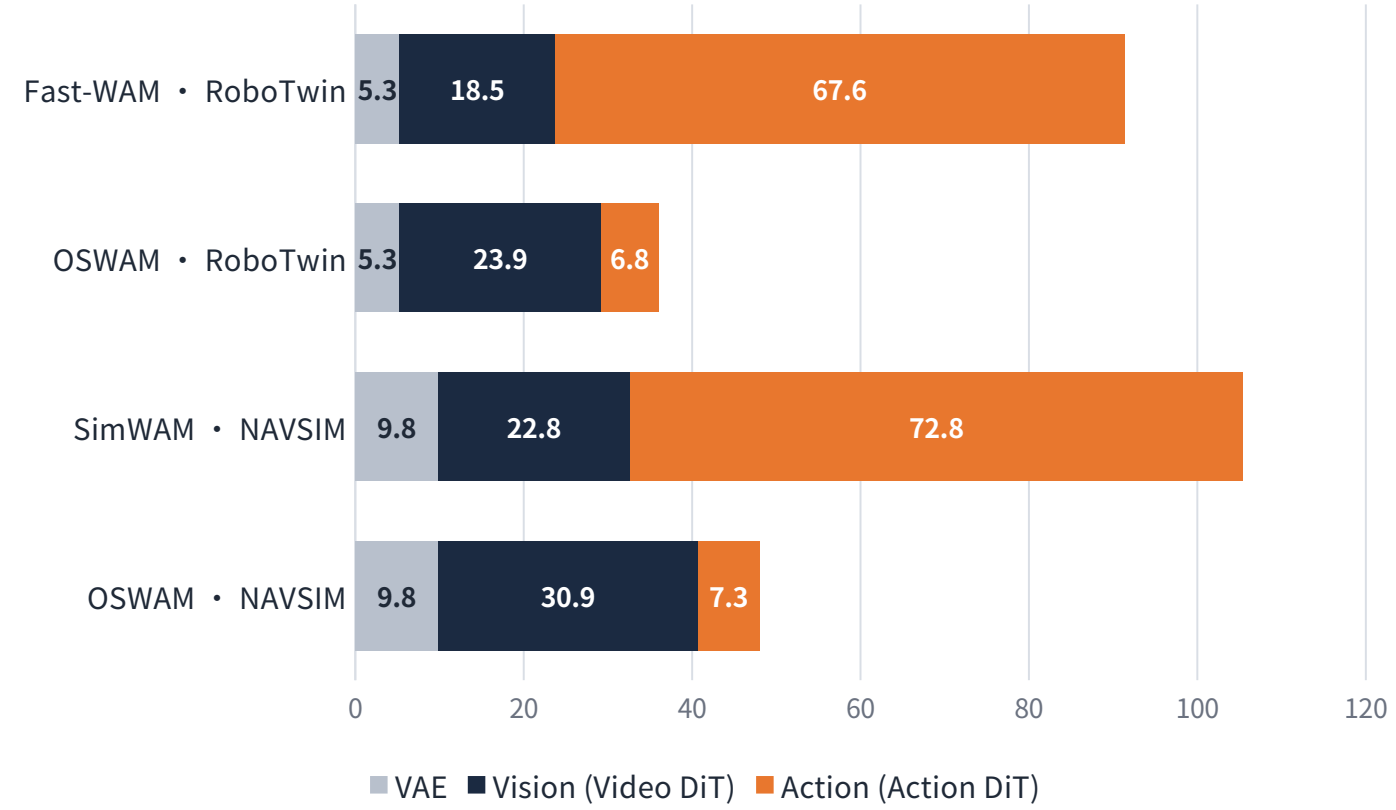
90.00% vs 76.40%



Source: Table 2 (50 tasks, Clean / Randomized avg.; H20)

# Action GPU time drops ~10×; imagination tokens add only ~5–8 ms

GPU time breakdown (ms)




| Model | Overall latency | Compiled |
|---|---|---|
| Fast-WAM | 323.67 | 73.64 |
| **OSWAM** | **95.35** | **40.81** |
| SimWAM | 330.8 | 90.53 |
| **OSWAM** | **99.70** | **52.64** |

Top two rows RoboTwin, bottom two NAVSIM

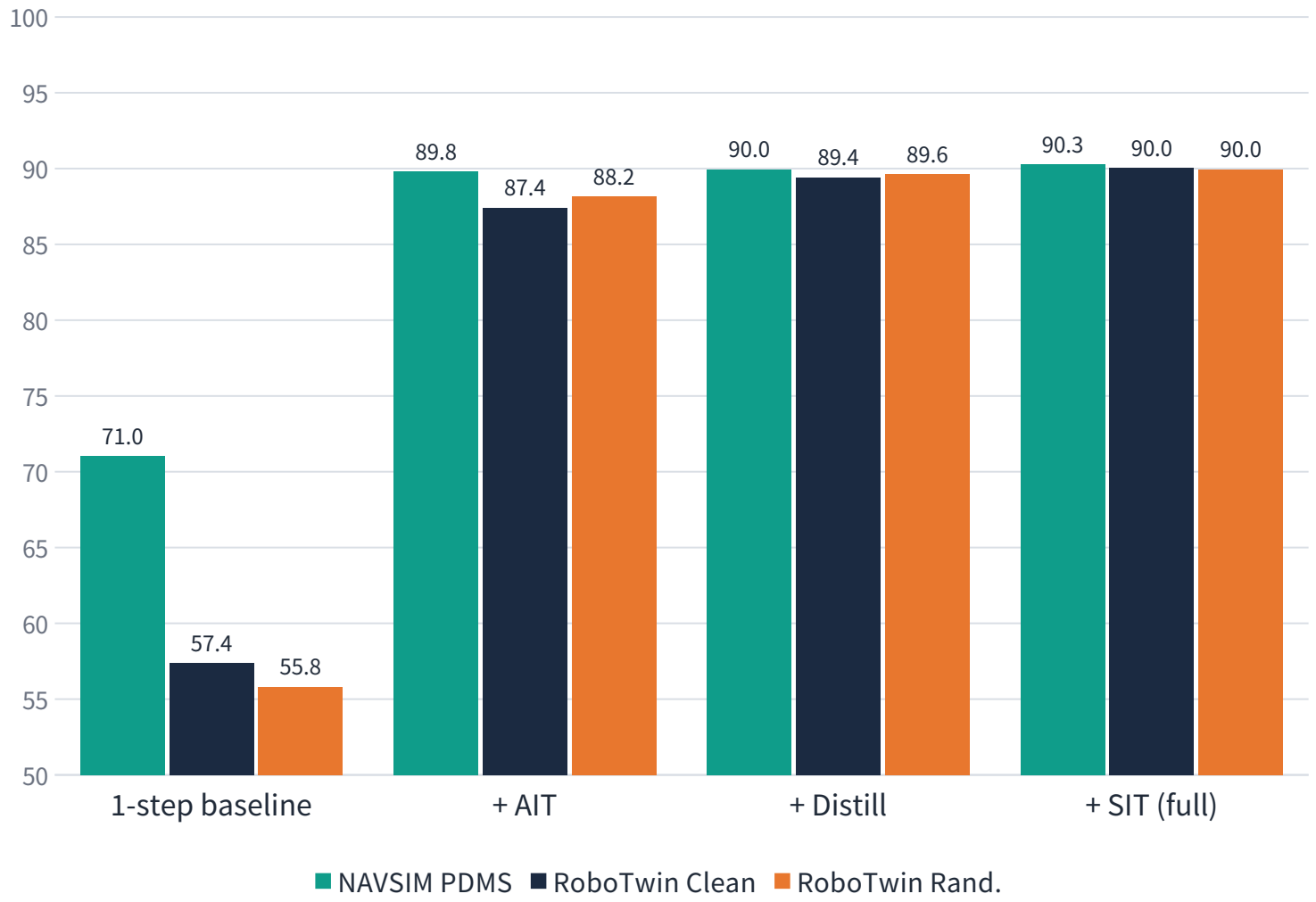
**3.39×**
**RoboTwin**
Overall latency

**3.32×**
**NAVSIM**
Overall latency

GPU components total ~36 ms; the rest of the 95 ms is CPU / scheduling overhead; torch.compile + CUDA Graph cut it to 40.81 ms. Peak memory 12.70 → 12.75 GB


Source: Table 3 (RTX 5090, cached language context, ms)

# AIT delivers nearly all gains; distillation and SIT add a little




**+18.8 / +30**

**+ AIT (structured start)**

NAVSIM PDMS / RoboTwin Clean; latency ~+27 ms

**+0.15 / +2.0**

**+ Privileged distillation**

RoboTwin Rand. +1.5; zero inference cost

**+0.32 / +0.6**

**+ SIT**

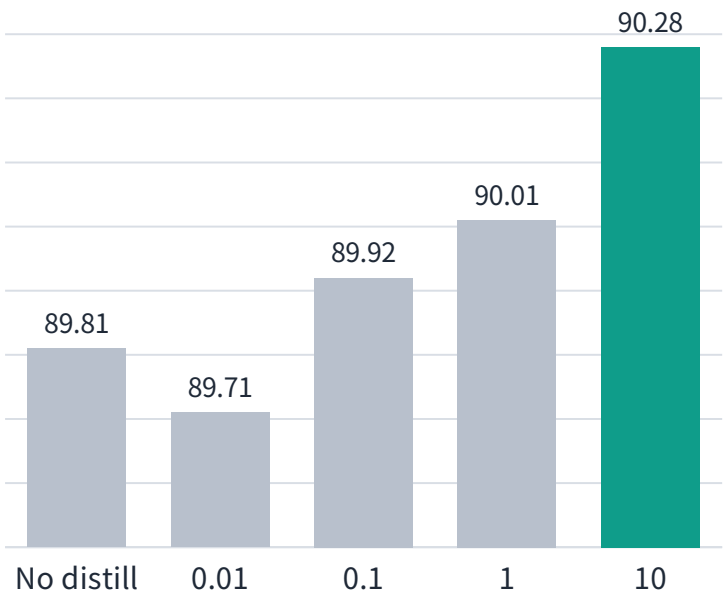
RoboTwin Rand. +0.3; latency +0.2–2.9 ms



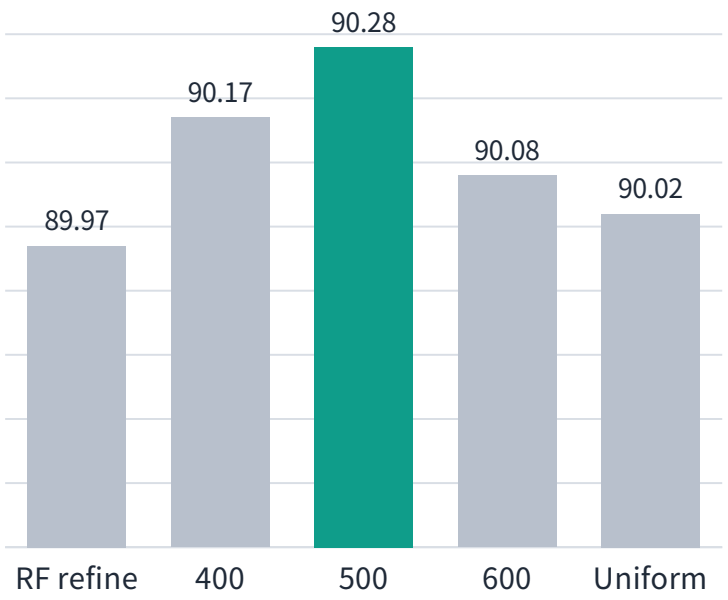
Source: Table 4 (baseline = 1-step Fast-WAM; H20 latency)

# Mid re-noising is best; near-noise teacher futures barely matter

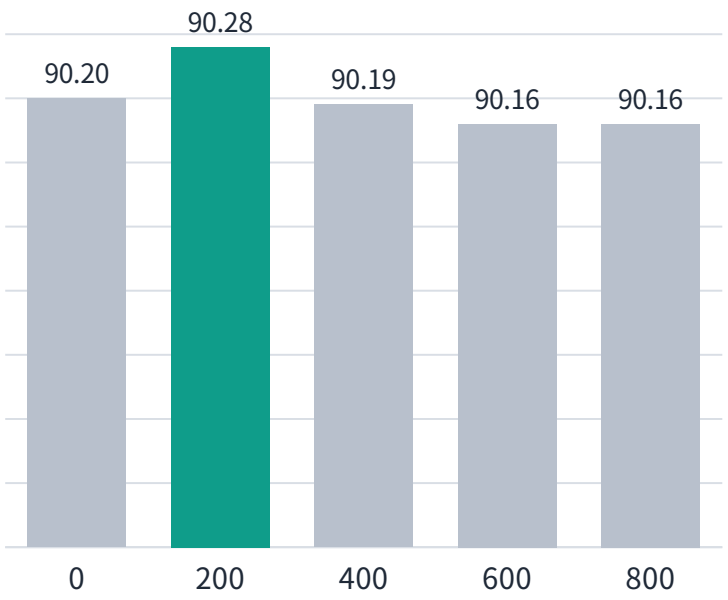
## Distill weight λ_dist




## Re-noise level τ_r




## Teacher video step τ_v




Only large weights help; λ = 0.01 is slightly below baseline

Mid noise (σ = 0.5) is best; fixed beats uniform sampling

800 (≈pure noise) still gives 90.16 ≈ clean 90.20

Note: most differences are 0.1–0.3 PDMS; no multi-seed variance is reported.



Source: Tables 5–7 (NAVSIM, PDMS)

# Best EPDMS on NAVSIM v2; top open-loop L2 and collision on nuScenes

## NAVSIM v2 （EPDMS ↑）

| Method | Action steps | EPDMS |
|---|---|---|
| Human Agent | – | 90.3 |
| DiffusionDrive | – | 84.5 |
| DriveVLA-W0 | – | 86.1 |
| Epona | 100 | 85.1 |
| DriveLaW | 5 | 88.6 |
| Metis | 10 | 89.5 |
| SimWAM | 10 | 90.2 |
| **OSWAM** | **1** | **90.3** |

## nuScenes open-loop planning (avg., ↓)

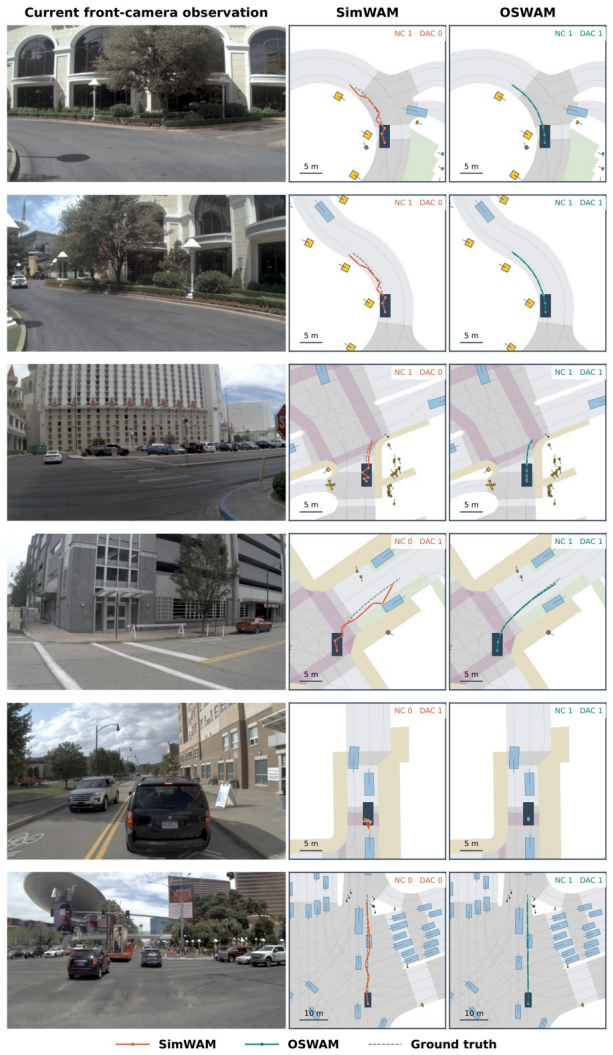
| Method | Aux. supervision | L2 (m) | Collision (%) |
|---|---|---|---|
| UniAD | Map/Box/Motion… | 1.03 | 0.31 |
| VAD-Base | Map/Box/Motion | 1.22 | 0.53 |
| GenAD | Map/Box/Motion | 0.91 | 0.43 |
| Epona | None | 1.25 | 0.36 |
| DriveLaW | None | 1.15 | 0.24 |
| PWM | Text | 0.41 | 0.04 |
| **OSWAM** | **None** | **0.41** | **0.03** |

**Reading:** on NAVSIM v2, 1-step OSWAM slightly beats 10-step methods; nuScenes is open-loop, useful for reference but not strong evidence of planning ability.



Source: Table A.1 (NAVSIM-v2 navtest), Table A.2 (nuScenes open-loop)

# 1 step each: SimWAM jitters off-road; OSWAM is smooth, compliant




## How to read this figure

- One scene per row: front camera | 1-step SimWAM | OSWAM
- Red = SimWAM, cyan = OSWAM, gray = ground truth
- Top right: NC (no at-fault collision) / DAC (drivable area compliance)

**68.9 → 90.3**

**1-step PDMS: SimWAM → OSWAM**

Table A.4

In turns and complex interactions, 1-step SimWAM leaves the drivable area or collides; OSWAM, also one step, yields smooth trajectories satisfying NC and DAC.



Source: Fig. B.1 (NAVSIM v1, 1-step SimWAM vs OSWAM)

# Real PiPER: slightly beats Fast-WAM, completes 22–34% faster

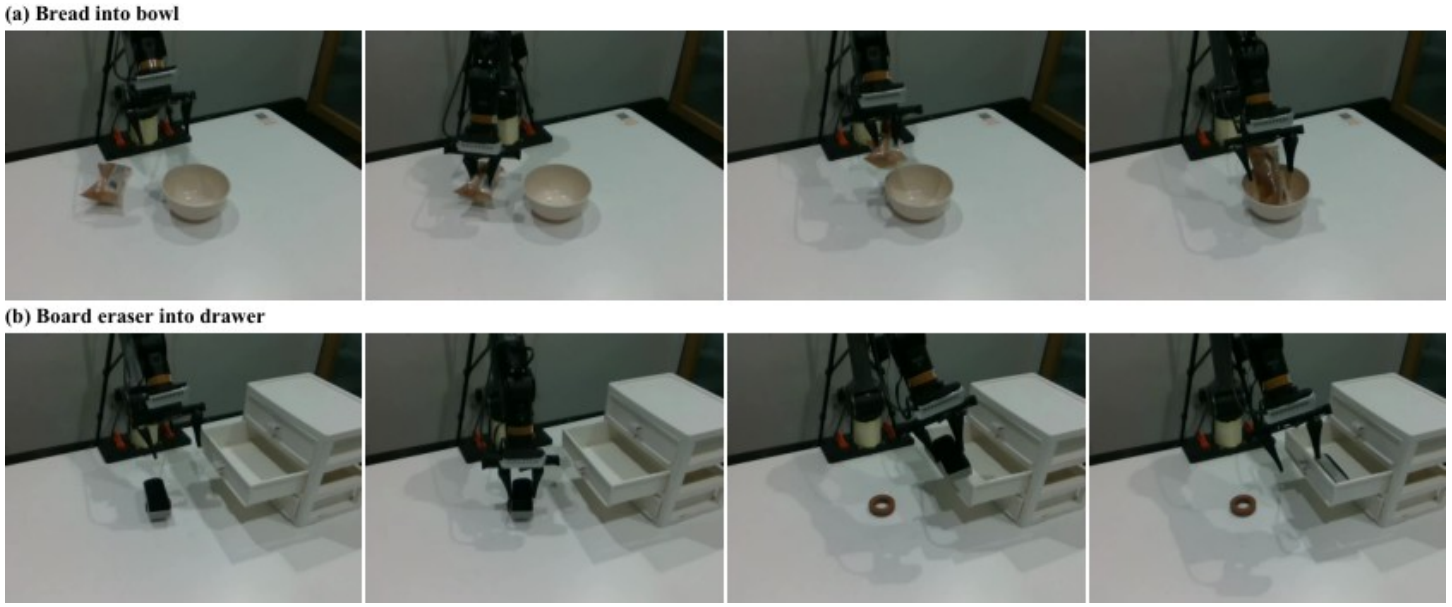



Fig. 5: (a) bread into bowl; (b) eraser into open drawer (avoiding drawer edge and cabinet)

| Task | Fast-WAM | OSWAM |
|---|---|---|
| Bread → bowl success | 80% | 85% |
| Eraser → drawer success | 75% | 80% |

**−34.2%**

**Bread task**

24.6 → 16.2 s

**−21.8%**

**Eraser task**

17.6 → 13.8 s

**Caveat**

Only 20 trials per task: 5 pts = 1 success, not statistically significant; shorter completion time (successful episodes, video-estimated) better shows the value of low latency.



Source: Table 8, Fig. 5 (50 demos, 20 trials per task)

# Six open questions

### 1. What “one step” means

Really “Video DiT regresses a coarse action + one Action DiT residual fix”; shared ε makes it near-deterministic; multimodal action distributions not discussed

### 2. Small distill gains

NAVSIM only +0.15 PDMS; teacher futures noised to $\tau = 800$ still give $90.16 \approx 90.20 \rightarrow$ “privileged future info” is doubtful; no variance

### 3. Not fully on par

RoboTwin 1.83 pts below Fast-WAM; matches or beats it on only 22/50 (Clean) and 15/50 (Rand.) tasks

### 4. Mixed latency setups

Main tables use H20, efficiency table RTX 5090; some baselines reproduced; 40 ms is compiled, not comparable to others’ uncompiled numbers

### 5. Small real-robot study

Only 20 trials per task, so 5 pts = 1 success; shorter completion time is more convincing

### 6. Prior only initializes

Action tokens do not attend to AIT / SIT; using scene context and privileged supervision in the Action DiT is unexplored (as authors admit)

Summary

# Simple, effective: let the world-model backbone seed the actions

**Problem**

Direct-policy WAMs already skip video generation; the remaining bottleneck is ~10 action denoising steps, rooted in a pure-noise start lacking action structure

**Method**

AIT / SIT yield a structured prior in one Video DiT pass → re-noised to $\sigma = 0.5$ → one-step Action DiT fix; a privileged teacher distills future information

**Results**

NAVSIM PDMS 90.3 (on par), RoboTwin 90.0% (−1.83); ~3× faster, 40–53 ms compiled, enabling >10 Hz control

**Takeaway:** the main gain comes from the "structured start" (AIT), an idea transferable to real-time deployment of other diffusion / flow policies. Distillation and SIT add little; their mechanisms need more rigorous validation.